

```

import pandas as pd

file_path = "C:/Users/Vinay/End-to-End E-commerce-Intelligence-System/dataset/"

#Load dataset
def load_dataset(sheet):
    df = pd.read_csv(f"{file_path}"+f"{sheet}")
    print(df.head())
    print(df.info())
    print(df.isnull().sum())
    return df

# Find primary and foreign key, duplicates in category_translation.
category = load_dataset("category_translation.csv")

print("Primary key is product_category_name. Doesnt have a foreign key")

print(category['product_category_name'].duplicated().sum())
print(category['product_category_name_english'].duplicated().sum())
print("There are no null or na in this table. There are no duplicates")
print("\n")

# Find primary and foreign key, duplicates in customers.
customers = load_dataset("customers.csv")

print("Primary key is customer_id")

print(customers['customer_id'].duplicated().sum())
print("There are no null or na in this table. Customer id is not duplicate")
print("\n")

# Find primary and foreign key, duplicates in location.csv.
location = load_dataset("location.csv")

print("Primary key is geolocation_zip_code_prefix")

print("There are no null or na in this table. Location columns can have duplicates")
print("\n")

# Find primary and foreign key, duplicates in order_item.csv.
order_item = load_dataset("order_item.csv")

print("Primary key is order_id. Foreign key is product_id, seller_id")

```

```

print("There are no null or na in this table.")

print(order_item['order_id'].duplicated().sum())
print("order id can be repeatative one order can have multiple
products")

# Change datatype of shipping_limit_date to datetime
order_item['shipping_limit_date'] =
pd.to_datetime(order_item['shipping_limit_date'])
print("\n")

# Find primary and foreign key, duplicates in orders.csv.
orders = load_dataset("orders.csv")

print("Primary key is order_id. Foreign key is customer_id")

print("Null or na are found in order_approved_at,
order_delivered_carrier_date and order_delivered_customer_date")

print(orders['order_id'].duplicated().sum())
print("There are no duplicates")

# Dropping order_approved_at and order_delivered_carrier_date
orders.drop(columns=['order_approved_at',
'order_delivered_carrier_date'], inplace=True)

# Change datatype of order_purchase_timestamp,
order_delivered_customer_date to datetime
orders['order_purchase_timestamp'] =
pd.to_datetime(orders['order_purchase_timestamp'], errors='coerce')
orders['order_delivered_customer_date'] =
pd.to_datetime(orders['order_delivered_customer_date'],
errors='coerce')
orders['order_estimated_delivery_date'] =
pd.to_datetime(orders['order_estimated_delivery_date'],
errors='coerce')

# Filling null/na order_delivered_customer_date with
order_estimated_delivery_date
orders['order_delivered_customer_date'].fillna("Unknown", inplace =
True)
print("\n")

# Find primary and foreign key, duplicates in payments.csv.
payments = load_dataset("payments.csv")

print("Primary key is order_id")

print("There are no null or na in this table.")

```

```

print(payments['order_id'].duplicated().sum())
print("Order id has duplicates")
payments.drop_duplicates(subset=['order_id'], inplace=True)
print("\n")

# Find primary and foreign key, duplicates in products.csv.
products = load_dataset("products.csv")

print("Primary key is product_id")

print("Null or na is found in product_category_name,
product_name_lenght, product_description_lenght, product_photos_qty, \
product_weight_g, product_length_cm, product_height_cm,
product_width_cm")

#Dropping product_category_name, product_name_lenght,
product_description_lenght and product_photos_qty
products.drop(columns=['product_name_lenght', 'product_description_leng
ht', 'product_photos_qty'], inplace = True)

# Filling product_weight_g, product_length_cm, product_height_cm,
product_width_cm with mode
products['product_weight_g'].fillna('Unknown', inplace=True)
products['product_length_cm'].fillna('Unknown', inplace=True)
products['product_height_cm'].fillna('Unknown', inplace=True)
products['product_width_cm'].fillna('Unknown', inplace=True)
products['product_category_name'].fillna('Unknown', inplace=True)

# Changing column names of product_name_lenght and
product_description_lenght
products.rename(columns={'product_name_lenght':
'product_name_length'}, inplace=True)
products.rename(columns={'product_description_lenght':
'product_description_length'}, inplace=True)
print("\n")

# Find primary and foreign key, duplicates in reviews.csv.
reviews = load_dataset("reviews.csv")

print("Primary key is review_id. Foreign key is order_id")

print("Null or na are found in review_comment_title,
review_comment_message. There are no duplicates")

# Dropping review_comment_title and review_comment_messageas there are
too many Nan in them.
reviews.drop(columns=['review_comment_title', 'review_comment_message']
, inplace = True)

# Change datatype of review_creation_date, review_answer_timestamp to

```

```

datetime
reviews['review_creation_date'] =
pd.to_datetime(reviews['review_creation_date'])
reviews['review_answer_timestamp'] =
pd.to_datetime(reviews['review_answer_timestamp'])
print("\n")

# Find primary and foreign key, duplicates in sellers.csv.
sellers = load_dataset("sellers.csv")

print("Primary key is seller_id")

print("There are no null or na in any columns. There are no
duplicates")

```

```

    product_category_name product_category_name_english
0          beleza_saude             health_beauty
1  informatica_acessorios      computers_accessories
2          automotivo             auto
3      cama_mesa_banho             bed_bath_table
4      moveis_decoracao      furniture_decor
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71 entries, 0 to 70
Data columns (total 2 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_category_name                71 non-null    object
1   product_category_name_english        71 non-null    object
dtypes: object(2)
memory usage: 1.2+ KB
None

```

```

product_category_name                0
product_category_name_english        0
dtype: int64
Primary key is product_category_name. Doesnt have a foreign key
0
0

```

There are no null or na in this table. There are no duplicates

|   | customer_id                      | customer_unique_id               |
|---|----------------------------------|----------------------------------|
| 0 | 06b8999e2fba1a1fbc88172c00ba8bc7 | 861eff4711a542e4b93843c6dd7febb0 |
| 1 | 18955e83d337fd6b2def6b18a428ac77 | 290c77bc529b7ac935b93aa66c333dc3 |
| 2 | 4e7b3e00288586ebd08712fdd0374a03 | 060e732b5b29e8181a18229c7b0b2b5e |
| 3 | b2b6027bc5c5109e529d4dc6358b12c3 | 259dac757896d24d7702b9acbbff3f3c |

```
4 4f2d8ab171c80ec8364f7c12e35b23ad 345ecd01c38d18a9036ed96c73b8d066
```

|   | customer_zip_code_prefix | customer_city         | customer_state |
|---|--------------------------|-----------------------|----------------|
| 0 | 14409                    | franca                | SP             |
| 1 | 9790                     | sao bernardo do campo | SP             |
| 2 | 1151                     | sao paulo             | SP             |
| 3 | 8775                     | mogi das cruze        | SP             |
| 4 | 13056                    | campinas              | SP             |

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 99441 entries, 0 to 99440
```

```
Data columns (total 5 columns):
```

| # | Column                   | Non-Null Count | Dtype  |
|---|--------------------------|----------------|--------|
| 0 | customer_id              | 99441 non-null | object |
| 1 | customer_unique_id       | 99441 non-null | object |
| 2 | customer_zip_code_prefix | 99441 non-null | int64  |
| 3 | customer_city            | 99441 non-null | object |
| 4 | customer_state           | 99441 non-null | object |

```
dtypes: int64(1), object(4)
```

```
memory usage: 3.8+ MB
```

```
None
```

|                          |   |
|--------------------------|---|
| customer_id              | 0 |
| customer_unique_id       | 0 |
| customer_zip_code_prefix | 0 |
| customer_city            | 0 |
| customer_state           | 0 |

```
dtype: int64
```

```
Primary key is customer_id
```

```
0
```

```
There are no null or na in this table.Customer id is not duplicate
```

|   | geolocation_zip_code_prefix | geolocation_lat | geolocation_lng | \ |
|---|-----------------------------|-----------------|-----------------|---|
| 0 | 1037                        | -23.545621      | -46.639292      |   |
| 1 | 1046                        | -23.546081      | -46.644820      |   |
| 2 | 1046                        | -23.546129      | -46.642951      |   |
| 3 | 1041                        | -23.544392      | -46.639499      |   |
| 4 | 1035                        | -23.541578      | -46.641607      |   |

|   | geolocation_city | geolocation_state |
|---|------------------|-------------------|
| 0 | sao paulo        | SP                |
| 1 | sao paulo        | SP                |
| 2 | sao paulo        | SP                |
| 3 | sao paulo        | SP                |
| 4 | sao paulo        | SP                |

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000163 entries, 0 to 1000162
```

```
Data columns (total 5 columns):
```

| # | Column | Non-Null Count | Dtype |
|---|--------|----------------|-------|
|---|--------|----------------|-------|

```

---  -----
0  geolocation_zip_code_prefix  1000163 non-null  int64
1  geolocation_lat             1000163 non-null  float64
2  geolocation_lng             1000163 non-null  float64
3  geolocation_city            1000163 non-null  object
4  geolocation_state           1000163 non-null  object
dtypes: float64(2), int64(1), object(2)
memory usage: 38.2+ MB
None
geolocation_zip_code_prefix    0
geolocation_lat                0
geolocation_lng                0
geolocation_city               0
geolocation_state              0
dtype: int64
Primary key is geolocation_zip_code_prefix
There are no null or na in this table. Location columns can have
duplicates

```

|   | order_id                          | order_item_id | \ |
|---|-----------------------------------|---------------|---|
| 0 | 00010242fe8c5a6d1ba2dd792cb16214  | 1             |   |
| 1 | 00018f77f2f0320c557190d7a144bdd3  | 1             |   |
| 2 | 000229ec398224ef6ca0657da4fc703e  | 1             |   |
| 3 | 00024acbcdff0a6daa1e931b038114c75 | 1             |   |
| 4 | 00042b26cf59d7ce69dfabb4e55b4fd9  | 1             |   |

|   | product_id                       | seller_id                        |
|---|----------------------------------|----------------------------------|
| 0 | 4244733e06e7ecb4970a6e2683c13e61 | 48436dade18ac8b2bce089ec2a041202 |
| 1 | e5f2d52b802189ee658865ca93d83a8f | dd7ddc04e1b6c2c614352b383efe2d36 |
| 2 | c777355d18b72b67abbeef9df44fd0fd | 5b51032eddd242adc84c38acab88f23d |
| 3 | 7634da152a4610f1595efa32f14722fc | 9d7a1d34a5052409006425275ba1c2b4 |
| 4 | ac6c3623068f30de03045865e4e10089 | df560393f3a51e74553ab94004ba5c87 |

|   | shipping_limit_date | price  | freight_value |
|---|---------------------|--------|---------------|
| 0 | 2017-09-19 09:45:35 | 58.90  | 13.29         |
| 1 | 2017-05-03 11:05:13 | 239.90 | 19.93         |
| 2 | 2018-01-18 14:48:30 | 199.00 | 17.87         |
| 3 | 2018-08-15 10:10:18 | 12.99  | 12.79         |
| 4 | 2017-02-13 13:57:51 | 199.90 | 18.14         |

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 112650 entries, 0 to 112649
```

```
Data columns (total 7 columns):
```

| # | Column | Non-Null Count | Dtype |
|---|--------|----------------|-------|
|---|--------|----------------|-------|

```

---
0  order_id          112650 non-null object
1  order_item_id     112650 non-null int64
2  product_id        112650 non-null object
3  seller_id         112650 non-null object
4  shipping_limit_date 112650 non-null object
5  price             112650 non-null float64
6  freight_value     112650 non-null float64

```

dtypes: float64(2), int64(1), object(4)

memory usage: 6.0+ MB

None

```

order_id          0
order_item_id     0
product_id        0
seller_id         0
shipping_limit_date 0
price             0
freight_value     0

```

dtype: int64

Primary key is order\_id. Foreign key is product\_id, seller\_id

There are no null or na in this table.

13984

order id can be repeatative one order can have multiple products

|   | order_id                         | customer_id                      |
|---|----------------------------------|----------------------------------|
| 0 | e481f51cbdc54678b7cc49136f2d6af7 | 9ef432eb6251297304e76186b10a928d |
| 1 | 53cdb2fc8bc7dce0b6741e2150273451 | b0830fb4747a6c6d20dea0b8c802d7ef |
| 2 | 47770eb9100c2d0c44946d9cf07ec65d | 41ce2a54c0b03bf3443c3d931a367089 |
| 3 | 949d5b44dbf5de918fe9c16f97b45f8a | f88197465ea7920adcdbec7375364d82 |
| 4 | ad21c59c0840e6cb83a9ceb5573f8159 | 8ab97904e6daea8866dbdbc4fb7aad2c |

|   | order_status | order_purchase_timestamp | order_approved_at   | \ |
|---|--------------|--------------------------|---------------------|---|
| 0 | delivered    | 2017-10-02 10:56:33      | 2017-10-02 11:07:15 |   |
| 1 | delivered    | 2018-07-24 20:41:37      | 2018-07-26 03:24:27 |   |
| 2 | delivered    | 2018-08-08 08:38:49      | 2018-08-08 08:55:23 |   |
| 3 | delivered    | 2017-11-18 19:28:06      | 2017-11-18 19:45:59 |   |
| 4 | delivered    | 2018-02-13 21:18:39      | 2018-02-13 22:20:29 |   |

|   | order_delivered_carrier_date | order_delivered_customer_date | \ |
|---|------------------------------|-------------------------------|---|
| 0 | 2017-10-04 19:55:00          | 2017-10-10 21:25:13           |   |
| 1 | 2018-07-26 14:31:00          | 2018-08-07 15:27:45           |   |
| 2 | 2018-08-08 13:50:00          | 2018-08-17 18:06:29           |   |
| 3 | 2017-11-22 13:39:59          | 2017-12-02 00:28:42           |   |

4                    2018-02-14 19:46:34                    2018-02-16 18:17:02

```
order_estimated_delivery_date
0      2017-10-18 00:00:00
1      2018-08-13 00:00:00
2      2018-09-04 00:00:00
3      2017-12-15 00:00:00
4      2018-02-26 00:00:00
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 99441 entries, 0 to 99440

Data columns (total 8 columns):

| # | Column                        | Non-Null Count | Dtype  |
|---|-------------------------------|----------------|--------|
| 0 | order_id                      | 99441 non-null | object |
| 1 | customer_id                   | 99441 non-null | object |
| 2 | order_status                  | 99441 non-null | object |
| 3 | order_purchase_timestamp      | 99441 non-null | object |
| 4 | order_approved_at             | 99281 non-null | object |
| 5 | order_delivered_carrier_date  | 97658 non-null | object |
| 6 | order_delivered_customer_date | 96476 non-null | object |
| 7 | order_estimated_delivery_date | 99441 non-null | object |

dtypes: object(8)

memory usage: 6.1+ MB

None

|                               |      |
|-------------------------------|------|
| order_id                      | 0    |
| customer_id                   | 0    |
| order_status                  | 0    |
| order_purchase_timestamp      | 0    |
| order_approved_at             | 160  |
| order_delivered_carrier_date  | 1783 |
| order_delivered_customer_date | 2965 |
| order_estimated_delivery_date | 0    |

dtype: int64

Primary key is order\_id. Foreign key is customer\_id

Null or na are found in order\_approved\_at,  
order\_delivered\_carrier\_date and order\_delivered\_customer\_date

0

There are no duplicates

C:\Users\Vinay\AppData\Local\Temp\ipykernel\_3236\3334369185.py:73:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
orders['order_delivered_customer_date'].fillna("Unknown", inplace =
True)
C:\Users\Vinay\AppData\Local\Temp\ipykernel_3236\3334369185.py:73:
FutureWarning: Setting an item of incompatible dtype is deprecated and
will raise an error in a future version of pandas. Value 'Unknown' has
dtype incompatible with datetime64[ns], please explicitly cast to a
compatible dtype first.
orders['order_delivered_customer_date'].fillna("Unknown", inplace =
True)
```

|   | order_id                         | payment_sequential | payment_type |
|---|----------------------------------|--------------------|--------------|
| 0 | b81ef226f3fe1789b1e8b2acac839d17 | 1                  | credit_card  |
| 1 | a9810da82917af2d9aefd1278f1dcfa0 | 1                  | credit_card  |
| 2 | 25e8ea4e93396b6fa0d3dd708e76c1bd | 1                  | credit_card  |
| 3 | ba78997921bbcdc1373bb41e913ab953 | 1                  | credit_card  |
| 4 | 42fdf880ba16b47b59251dd489d4441a | 1                  | credit_card  |

|   | payment_installments | payment_value |
|---|----------------------|---------------|
| 0 | 8                    | 99.33         |
| 1 | 1                    | 24.39         |
| 2 | 1                    | 65.71         |
| 3 | 8                    | 107.78        |
| 4 | 2                    | 128.45        |

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103886 entries, 0 to 103885
Data columns (total 5 columns):
```

| # | Column               | Non-Null Count  | Dtype   |
|---|----------------------|-----------------|---------|
| 0 | order_id             | 103886 non-null | object  |
| 1 | payment_sequential   | 103886 non-null | int64   |
| 2 | payment_type         | 103886 non-null | object  |
| 3 | payment_installments | 103886 non-null | int64   |
| 4 | payment_value        | 103886 non-null | float64 |

```
dtypes: float64(1), int64(2), object(2)
```

```
memory usage: 4.0+ MB
```

```
None
```

|                      |   |
|----------------------|---|
| order_id             | 0 |
| payment_sequential   | 0 |
| payment_type         | 0 |
| payment_installments | 0 |

```

payment_value      0
dtype: int64
Primary key is order_id
There are no null or na in this table.
4446
Order id has duplicates

```

```

      product_id  product_category_name \
0  1e9e8ef04dbcff4541ed26657ea517e5      perfumaria
1  3aa071139cb16b67ca9e5dea641aaa2f      artes
2  96bd76ec8810374ed1b65e291975717f  esporte_lazer
3  cef67bcfe19066a932b7673e239eb23d      bebes
4  9dc1a7de274444849c219cff195d0b71  utilidades_domesticas

```

```

      product_name_lenght  product_description_lenght  product_photos_qty
\
0                40.0                287.0                1.0
1                44.0                276.0                1.0
2                46.0                250.0                1.0
3                27.0                261.0                1.0
4                37.0                402.0                4.0

```

```

      product_weight_g  product_length_cm  product_height_cm
product_width_cm
0                225.0                16.0                10.0
14.0
1                1000.0                30.0                18.0
20.0
2                154.0                18.0                9.0
15.0
3                371.0                26.0                4.0
26.0
4                625.0                20.0                17.0
13.0

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 32951 entries, 0 to 32950
```

```
Data columns (total 9 columns):
```

| # | Column                     | Non-Null Count | Dtype   |
|---|----------------------------|----------------|---------|
| 0 | product_id                 | 32951 non-null | object  |
| 1 | product_category_name      | 32341 non-null | object  |
| 2 | product_name_lenght        | 32341 non-null | float64 |
| 3 | product_description_lenght | 32341 non-null | float64 |
| 4 | product_photos_qty         | 32341 non-null | float64 |

|   |                   |       |          |         |
|---|-------------------|-------|----------|---------|
| 5 | product_weight_g  | 32949 | non-null | float64 |
| 6 | product_length_cm | 32949 | non-null | float64 |
| 7 | product_height_cm | 32949 | non-null | float64 |
| 8 | product_width_cm  | 32949 | non-null | float64 |

dtypes: float64(7), object(2)

memory usage: 2.3+ MB

None

product\_id 0

product\_category\_name 610

product\_name\_lenght 610

product\_description\_lenght 610

product\_photos\_qty 610

product\_weight\_g 2

product\_length\_cm 2

product\_height\_cm 2

product\_width\_cm 2

dtype: int64

Primary key is product\_id

Null or na is found in product\_category\_name, product\_name\_lenght, product\_description\_lenght, product\_photos\_qty, product\_weight\_g, product\_length\_cm, product\_height\_cm, product\_width\_cm

C:\Users\Vinay\AppData\Local\Temp\ipykernel\_3236\3334369185.py:100:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
products['product_weight_g'].fillna('Unknown', inplace=True)
```

C:\Users\Vinay\AppData\Local\Temp\ipykernel\_3236\3334369185.py:100:

FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'Unknown' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```
products['product_weight_g'].fillna('Unknown', inplace=True)
```

C:\Users\Vinay\AppData\Local\Temp\ipykernel\_3236\3334369185.py:101:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
products['product_length_cm'].fillna('Unknown', inplace=True)
C:\Users\Vinay\AppData\Local\Temp\ipykernel_3236\3334369185.py:101:
FutureWarning: Setting an item of incompatible dtype is deprecated and
will raise an error in a future version of pandas. Value 'Unknown' has
dtype incompatible with float64, please explicitly cast to a
compatible dtype first.
```

```
products['product_length_cm'].fillna('Unknown', inplace=True)
C:\Users\Vinay\AppData\Local\Temp\ipykernel_3236\3334369185.py:102:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
products['product_height_cm'].fillna('Unknown', inplace=True)
C:\Users\Vinay\AppData\Local\Temp\ipykernel_3236\3334369185.py:102:
FutureWarning: Setting an item of incompatible dtype is deprecated and
will raise an error in a future version of pandas. Value 'Unknown' has
dtype incompatible with float64, please explicitly cast to a
compatible dtype first.
```

```
products['product_height_cm'].fillna('Unknown', inplace=True)
C:\Users\Vinay\AppData\Local\Temp\ipykernel_3236\3334369185.py:103:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
products['product_width_cm'].fillna('Unknown', inplace=True)
C:\Users\Vinay\AppData\Local\Temp\ipykernel_3236\3334369185.py:103:
FutureWarning: Setting an item of incompatible dtype is deprecated and
```

will raise an error in a future version of pandas. Value 'Unknown' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```
products['product_width_cm'].fillna('Unknown', inplace=True)
```

C:\Users\Vinay\AppData\Local\Temp\ipykernel\_3236\3334369185.py:104:  
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.  
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
products['product_category_name'].fillna('Unknown', inplace=True)
```

|   | review_id                        | order_id                         |
|---|----------------------------------|----------------------------------|
| 0 | 7bc2406110b926393aa56f80a40eba40 | 73fc7af87114b39712e6da79b0a377eb |
| 1 | 80e641a11e56f04c1ad469d5645fdfde | a548910a1c6147796b98fdf73dbeba33 |
| 2 | 228ce5500dc1d8e020d8d1322874b6f0 | f9e4b658b201a9f2ecdecbb34bed034b |
| 3 | e64fb393e7b32834bb789ff8bb30750e | 658677c97b385a9be170737859d3511b |
| 4 | f7c4243c7fe1938f181bec41a392bdeb | 8e6bfb81e283fa7e4f11123a3fb894f1 |

|   | review_score | review_comment_title |
|---|--------------|----------------------|
| 0 | 4            | NaN                  |
| 1 | 5            | NaN                  |
| 2 | 5            | NaN                  |
| 3 | 5            | NaN                  |
| 4 | 5            | NaN                  |

|   | review_creation_date | review_comment_message                                       |
|---|----------------------|--------------------------------------------------------------|
| 0 | 00:00:00             | NaN 2018-01-18                                               |
| 1 | 00:00:00             | NaN 2018-03-10                                               |
| 2 | 00:00:00             | NaN 2018-02-17                                               |
| 3 | 00:00:00             | Recebi bem antes do prazo estipulado. 2017-04-21             |
| 4 | 00:00:00             | Parabéns lojas lannister adorei comprar pela I... 2018-03-01 |

00:00:00

review\_answer\_timestamp

0 2018-01-18 21:46:59

1 2018-03-11 03:05:13

2 2018-02-18 14:36:24

3 2017-04-21 22:02:06

4 2018-03-02 10:26:53

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 99224 entries, 0 to 99223

Data columns (total 7 columns):

| # | Column                  | Non-Null Count | Dtype  |
|---|-------------------------|----------------|--------|
| 0 | review_id               | 99224 non-null | object |
| 1 | order_id                | 99224 non-null | object |
| 2 | review_score            | 99224 non-null | int64  |
| 3 | review_comment_title    | 11568 non-null | object |
| 4 | review_comment_message  | 40977 non-null | object |
| 5 | review_creation_date    | 99224 non-null | object |
| 6 | review_answer_timestamp | 99224 non-null | object |

dtypes: int64(1), object(6)

memory usage: 5.3+ MB

None

review\_id 0

order\_id 0

review\_score 0

review\_comment\_title 87656

review\_comment\_message 58247

review\_creation\_date 0

review\_answer\_timestamp 0

dtype: int64

Primary key is review\_id. Foreign key is order\_id

Null or na are found in review\_comment\_title, review\_comment\_message.

There are no duplicates

|   | seller_id                        | seller_zip_code_prefix | \ |
|---|----------------------------------|------------------------|---|
| 0 | 3442f8959a84dea7ee197c632cb2df15 | 13023                  |   |
| 1 | d1b65fc7debc3361ea86b5f14c68d2e2 | 13844                  |   |
| 2 | ce3ad9de960102d0677a81f5d0bb7b2d | 20031                  |   |
| 3 | c0f3eea2e14555b6faeea3dd58c1b1c3 | 4195                   |   |
| 4 | 51a04a8a6bdcb23deccc82b0b80742cf | 12914                  |   |

seller\_city seller\_state

0 campinas SP

1 mogi guacu SP

2 rio de janeiro RJ

3 sao paulo SP

4 braganca paulista SP

<class 'pandas.core.frame.DataFrame'>

```

RangeIndex: 3095 entries, 0 to 3094
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   seller_id              3095 non-null   object
1   seller_zip_code_prefix 3095 non-null   int64
2   seller_city            3095 non-null   object
3   seller_state           3095 non-null   object
dtypes: int64(1), object(3)
memory usage: 96.8+ KB
None
seller_id              0
seller_zip_code_prefix 0
seller_city            0
seller_state           0
dtype: int64
Primary key is seller_id
There are no null or na in any columns. There are no duplicates

# Validate data types and ranges and convert date columns to datetime
format
def validate(df):
    for col in df.columns:
        name = [n for n, v in globals().items() if v is df][0]
        print(f"\nTable: {name}")

        if df[col].dtype == 'object':
            print(f"Unique values in {col}: {df[col].nunique()}")
        elif df[col].dtype in ['int64', 'float64', 'datetime64[ns]']:
            print(f>Data type of {col}: {df[col].dtype}")
            print(f'Max value in {col}: {df[col].max()}')
            print(f'Min value in {col}: {df[col].min()}')

validate(orders)
validate(category)
validate(customers)
validate(location)
validate(order_item)
validate(payments)
validate(products)
validate(reviews)
validate(sellers)

Table: orders
Unique values in order_id: 99441

Table: orders
Unique values in customer_id: 99441

Table: orders

```

Unique values in order\_status: 8

Table: orders

Data type of order\_purchase\_timestamp: datetime64[ns]

Max value in order\_purchase\_timestamp: 2018-10-17 17:30:18

Min value in order\_purchase\_timestamp: 2016-09-04 21:15:19

Table: orders

Unique values in order\_delivered\_customer\_date: 95665

Table: orders

Data type of order\_estimated\_delivery\_date: datetime64[ns]

Max value in order\_estimated\_delivery\_date: 2018-11-12 00:00:00

Min value in order\_estimated\_delivery\_date: 2016-09-30 00:00:00

Table: category

Unique values in product\_category\_name: 71

Table: category

Unique values in product\_category\_name\_english: 71

Table: customers

Unique values in customer\_id: 99441

Table: customers

Unique values in customer\_unique\_id: 96096

Table: customers

Data type of customer\_zip\_code\_prefix: int64

Max value in customer\_zip\_code\_prefix: 99990

Min value in customer\_zip\_code\_prefix: 1003

Table: customers

Unique values in customer\_city: 4119

Table: customers

Unique values in customer\_state: 27

Table: location

Data type of geolocation\_zip\_code\_prefix: int64

Max value in geolocation\_zip\_code\_prefix: 99990

Min value in geolocation\_zip\_code\_prefix: 1001

Table: location

Data type of geolocation\_lat: float64

Max value in geolocation\_lat: 45.06593318269697

Min value in geolocation\_lat: -36.6053744107061

Table: location

Data type of geolocation\_lng: float64

Max value in geolocation\_lng: 121.10539381057764  
Min value in geolocation\_lng: -101.46676644931476

Table: location  
Unique values in geolocation\_city: 8011

Table: location  
Unique values in geolocation\_state: 27

Table: order\_item  
Unique values in order\_id: 98666

Table: order\_item  
Data type of order\_item\_id: int64  
Max value in order\_item\_id: 21  
Min value in order\_item\_id: 1

Table: order\_item  
Unique values in product\_id: 32951

Table: order\_item  
Unique values in seller\_id: 3095

Table: order\_item  
Data type of shipping\_limit\_date: datetime64[ns]  
Max value in shipping\_limit\_date: 2020-04-09 22:35:08  
Min value in shipping\_limit\_date: 2016-09-19 00:15:34

Table: order\_item  
Data type of price: float64  
Max value in price: 6735.0  
Min value in price: 0.85

Table: order\_item  
Data type of freight\_value: float64  
Max value in freight\_value: 409.68  
Min value in freight\_value: 0.0

Table: payments  
Unique values in order\_id: 99440

Table: payments  
Data type of payment\_sequential: int64  
Max value in payment\_sequential: 27  
Min value in payment\_sequential: 1

Table: payments  
Unique values in payment\_type: 5

Table: payments

Data type of payment\_installments: int64  
Max value in payment\_installments: 24  
Min value in payment\_installments: 0

Table: payments  
Data type of payment\_value: float64  
Max value in payment\_value: 13664.08  
Min value in payment\_value: 0.0

Table: products  
Unique values in product\_id: 32951

Table: products  
Unique values in product\_category\_name: 74

Table: products  
Unique values in product\_weight\_g: 2205

Table: products  
Unique values in product\_length\_cm: 100

Table: products  
Unique values in product\_height\_cm: 103

Table: products  
Unique values in product\_width\_cm: 96

Table: reviews  
Unique values in review\_id: 98410

Table: reviews  
Unique values in order\_id: 98673

Table: reviews  
Data type of review\_score: int64  
Max value in review\_score: 5  
Min value in review\_score: 1

Table: reviews  
Data type of review\_creation\_date: datetime64[ns]  
Max value in review\_creation\_date: 2018-08-31 00:00:00  
Min value in review\_creation\_date: 2016-10-02 00:00:00

Table: reviews  
Data type of review\_answer\_timestamp: datetime64[ns]  
Max value in review\_answer\_timestamp: 2018-10-29 12:27:35  
Min value in review\_answer\_timestamp: 2016-10-07 18:32:28

Table: sellers  
Unique values in seller\_id: 3095

```
Table: sellers
Data type of seller_zip_code_prefix: int64
Max value in seller_zip_code_prefix: 99730
Min value in seller_zip_code_prefix: 1001
```

```
Table: sellers
Unique values in seller_city: 611
```

```
Table: sellers
Unique values in seller_state: 23
```

```
# joining orders and customers
```

```
consolidated_dataset = orders.merge(customers, how='left',
on='customer_id')
consolidated_dataset = consolidated_dataset.merge(order_item,
how='left', on='order_id')
consolidated_dataset = consolidated_dataset.merge(products,
how='left', on='product_id')
consolidated_dataset = consolidated_dataset.merge(payments,
how='left', on='order_id')
consolidated_dataset = consolidated_dataset.merge(reviews, how='left',
on='order_id')
consolidated_dataset = consolidated_dataset.merge(sellers, how='left',
on='seller_id')
consolidated_dataset = consolidated_dataset.merge(category,
how='left', on='product_category_name')
consolidated_dataset['order_delivered_customer_date'] =
pd.to_datetime(consolidated_dataset['order_delivered_customer_date'],
errors='coerce')
print(consolidated_dataset.head().info())
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 33 columns):
```

| #  | Column                        | Non-Null Count | Dtype          |
|----|-------------------------------|----------------|----------------|
| 0  | order_id                      | 5 non-null     | object         |
| 1  | customer_id                   | 5 non-null     | object         |
| 2  | order_status                  | 5 non-null     | object         |
| 3  | order_purchase_timestamp      | 5 non-null     | datetime64[ns] |
| 4  | order_delivered_customer_date | 5 non-null     | datetime64[ns] |
| 5  | order_estimated_delivery_date | 5 non-null     | datetime64[ns] |
| 6  | customer_unique_id            | 5 non-null     | object         |
| 7  | customer_zip_code_prefix      | 5 non-null     | int64          |
| 8  | customer_city                 | 5 non-null     | object         |
| 9  | customer_state                | 5 non-null     | object         |
| 10 | order_item_id                 | 5 non-null     | float64        |
| 11 | product_id                    | 5 non-null     | object         |
| 12 | seller_id                     | 5 non-null     | object         |

```

13 shipping_limit_date      5 non-null    datetime64[ns]
14 price                    5 non-null    float64
15 freight_value            5 non-null    float64
16 product_category_name    5 non-null    object
17 product_weight_g          5 non-null    object
18 product_length_cm         5 non-null    object
19 product_height_cm         5 non-null    object
20 product_width_cm          5 non-null    object
21 payment_sequential        5 non-null    float64
22 payment_type              5 non-null    object
23 payment_installments      5 non-null    float64
24 payment_value             5 non-null    float64
25 review_id                 5 non-null    object
26 review_score              5 non-null    float64
27 review_creation_date      5 non-null    datetime64[ns]
28 review_answer_timestamp   5 non-null    datetime64[ns]
29 seller_zip_code_prefix    5 non-null    float64
30 seller_city                5 non-null    object
31 seller_state              5 non-null    object
32 product_category_name_english 5 non-null    object
dtypes: datetime64[ns](6), float64(8), int64(1), object(18)
memory usage: 1.4+ KB
None

```

### *# Feature engineering*

#### *# Total order value*

```

total_order_value = consolidated_dataset.groupby('order_id')
['payment_value'].sum().reset_index()
total_order_value.columns = ['order_id', 'total_order_value']

```

#### *# Delivery time*

```

consolidated_dataset['delivery_time'] =
(consolidated_dataset['order_delivered_customer_date'] -
consolidated_dataset['order_purchase_timestamp']).dt.days
average_delivery_time_per_customer =
consolidated_dataset.groupby('customer_unique_id')
['delivery_time'].mean().reset_index()
average_delivery_time_per_customer.columns = ['customer_unique_id',
'average_delivery_time']
consolidated_dataset['average_delivery_time'] =
average_delivery_time_per_customer['average_delivery_time']

```

#### *# Number of items per order*

```

items_per_order = consolidated_dataset.groupby('order_id')
['product_id'].count().reset_index()
items_per_order.columns = ['order_id', 'items_per_order']

```

#### *# Customer purchase frequency*

```

customer_purchase_frequency =

```

```

consolidated_dataset.groupby('customer_unique_id')
['order_id'].nunique().reset_index()
customer_purchase_frequency.columns = ['customer_unique_id',
'customer_purchase_frequency']
customer_purchase_frequency_delivery_time =
customer_purchase_frequency.merge(average_delivery_time_per_customer,
on='customer_unique_id')

# Customer Lifetime Value
customer_lifetime_value =
consolidated_dataset.groupby('customer_unique_id')
['payment_value'].sum().reset_index()
customer_lifetime_value.columns = ['customer_unique_id',
'customer_lifetime_value']

# Average order value per customer
customer_360 =
customer_lifetime_value.merge(customer_purchase_frequency_delivery_time, on='customer_unique_id')
customer_360['avg_order_value_per_customer'] =
customer_360['customer_lifetime_value'] /
customer_360['customer_purchase_frequency']

order_360 = total_order_value.merge(items_per_order, on = 'order_id')

print("****order_360****")
print(order_360)
print("****customer_360****")
print(customer_360)

****order_360****

```

|                 | order_id                         | total_order_value |
|-----------------|----------------------------------|-------------------|
| items_per_order |                                  |                   |
| 0               | 00010242fe8c5a6d1ba2dd792cb16214 | 72.19             |
| 1               |                                  |                   |
| 1               | 00018f77f2f0320c557190d7a144bdd3 | 259.83            |
| 1               |                                  |                   |
| 2               | 000229ec398224ef6ca0657da4fc703e | 216.87            |
| 1               |                                  |                   |
| 3               | 00024acbcdf0a6daa1e931b038114c75 | 25.78             |
| 1               |                                  |                   |
| 4               | 00042b26cf59d7ce69dfabb4e55b4fd9 | 218.04            |
| 1               |                                  |                   |
| ...             | ...                              | ...               |
| ...             |                                  |                   |
| 99436           | fffc94f6ce00a00581880bf54a75a037 | 343.40            |
| 1               |                                  |                   |
| 99437           | fffc46ef2263f404302a634eb57f7eb  | 386.53            |
| 1               |                                  |                   |

|       |                                  |        |
|-------|----------------------------------|--------|
| 99438 | fffce4705a9662cd70adb13d4a31832d | 116.85 |
| 1     |                                  |        |
| 99439 | fffe18544ffabc95dfada21779c9644f | 64.71  |
| 1     |                                  |        |
| 99440 | fffe41c64501cc87c801fd61db3f6244 | 55.79  |
| 1     |                                  |        |

[99441 rows x 3 columns]

\*\*\*\*customer\_360\*\*\*\*

|       | customer_unique_id               | customer_lifetime_value \ |
|-------|----------------------------------|---------------------------|
| 0     | 0000366f3b9a7992bf8c76cfd3221e2  | 141.90                    |
| 1     | 0000b849f77a49e4a4ce2b2a4ca5be3f | 27.19                     |
| 2     | 0000f46a3911fa3c0805444483337064 | 86.22                     |
| 3     | 0000f6ccb0745a6a4b88665a16c9f078 | 43.62                     |
| 4     | 0004aac84e0df4da2b147fca70cf8255 | 196.89                    |
| ...   | ...                              | ...                       |
| 96091 | fffcf5a5ff07b0908bd4e2dbc735a684 | 4134.84                   |
| 96092 | fffea47cd6d3cc0a88bd621562a9d061 | 84.58                     |
| 96093 | ffff371b4d645b6ecea244b27531430a | 112.46                    |
| 96094 | ffff5962728ec6157033ef9805bacc48 | 133.69                    |
| 96095 | ffffd2657e2aad2907e67c3e9daecbeb | 71.56                     |

|       | customer_purchase_frequency | average_delivery_time \ |
|-------|-----------------------------|-------------------------|
| 0     | 1                           | 6.0                     |
| 1     | 1                           | 3.0                     |
| 2     | 1                           | 25.0                    |
| 3     | 1                           | 20.0                    |
| 4     | 1                           | 13.0                    |
| ...   | ...                         | ...                     |
| 96091 | 1                           | 27.0                    |
| 96092 | 1                           | 30.0                    |
| 96093 | 1                           | 14.0                    |
| 96094 | 1                           | 11.0                    |
| 96095 | 1                           | 7.0                     |

|       | avg_order_value_per_customer |
|-------|------------------------------|
| 0     | 141.90                       |
| 1     | 27.19                        |
| 2     | 86.22                        |
| 3     | 43.62                        |
| 4     | 196.89                       |
| ...   | ...                          |
| 96091 | 4134.84                      |
| 96092 | 84.58                        |
| 96093 | 112.46                       |
| 96094 | 133.69                       |
| 96095 | 71.56                        |

[96096 rows x 5 columns]

```

# Exploratory Data Analysis

# New vs repeat customers
repeat_customers = (customer_360['customer_purchase_frequency'] >
1).sum()
new_customers = (customer_360['customer_purchase_frequency'] ==
1).sum()
new_vs_repeat = new_customers/repeat_customers
print(new_vs_repeat)
print("New customers are 3106% more than the repeat customers")

# High-value vs low-value customers
# High value customers are those with top 10% in metrics
top_10 = customer_360['customer_lifetime_value'].quantile(0.9)
high_value_customers = (customer_360['customer_lifetime_value'] >
top_10).sum()
low_value_customers = (customer_360['customer_lifetime_value'] <
top_10).sum()
high_vs_low = high_value_customers/low_value_customers
print("High value customers are 11% more than low value customers")

# Geographic distribution of customers
geographic_dist = consolidated_dataset.groupby('customer_state')
['customer_unique_id'].nunique()
print(consolidated_dataset)

# Monthly revenue trends
revenue_trends =
consolidated_dataset.groupby(consolidated_dataset['order_purchase_time
stamp'].dt.month_name())['payment_value'].sum()
print(revenue_trends)

# Order volume trends
order_volume_trends =
consolidated_dataset.groupby(consolidated_dataset['order_purchase_time
stamp'].dt.month_name())['order_id'].nunique()
print(order_volume_trends)

# Peak sales periods
peak_sales_periods =
consolidated_dataset.groupby(consolidated_dataset['order_purchase_time
stamp'].dt.to_period('M'))['order_id'].nunique()
print(peak_sales_periods)

# Top-selling product categories
top_selling_product =
(consolidated_dataset['product_category_name_english'].value_counts())
.sort_values(ascending=False)
print(top_selling_product)

```

```

# Revenue contribution by category
revenue_contribution_category =
consolidated_dataset.groupby('product_category_name_english')
['payment_value'].sum()
print(revenue_contribution_category)

# Product demand distribution
product_demand_distribution =
consolidated_dataset.groupby('product_category_name_english')
['order_item_id'].sum().astype(int)
print(product_demand_distribution)

# Top-performing sellers
top_sellers = consolidated_dataset.groupby('seller_id')
['payment_value'].sum().sort_values(ascending=False)
print(top_sellers.head(10))

# Seller contribution to revenue
total_payment = consolidated_dataset['payment_value'].sum()
seller_contribution = (top_sellers / total_payment) * 100
print(seller_contribution.head(10))

# Seller distribution
seller_distribution = pd.concat([top_sellers, seller_contribution],
axis=0)

# Distribution of review scores
review_scores_distribution =
consolidated_dataset['review_score'].value_counts().sort_index()

# Relationship between delivery time and ratings
delivery_time_vs_ratings = consolidated_dataset[['delivery_time',
'review_score']].corr().iloc[0,1]
print(delivery_time_vs_ratings)

# Identification of dissatisfaction patterns
dissatisfaction_patterns =
consolidated_dataset.groupby('delivery_time')['review_score'].mean()
print(dissatisfaction_patterns)

```

31.064064064064063

New customers are 3106% more than the repeat customers

High value customers are 11% more than low value customers

```

order_id
customer_id \
0          e481f51cbdc54678b7cc49136f2d6af7
9ef432eb6251297304e76186b10a928d
1          53cdb2fc8bc7dce0b6741e2150273451
b0830fb4747a6c6d20dea0b8c802d7ef
2          47770eb9100c2d0c44946d9cf07ec65d

```

41ce2a54c0b03bf3443c3d931a367089  
3 949d5b44dbf5de918fe9c16f97b45f8a  
f88197465ea7920adcdbec7375364d82  
4 ad21c59c0840e6cb83a9ceb5573f8159  
8ab97904e6daea8866dbdbc4fb7aad2c

...  
...  
114087 63943bddc261676b46f01ca7ac2f7bd8  
1fca14ff2861355f6e5f14306ff977a7  
114088 83c1379a015df1e13d02aae0204711ab  
1aa71eb042121263aafbe80c1b562c9c  
114089 11c177c8e97725db2631073c19f07b62  
b331b74b18dc79bcd6f6532d51e1637c1  
114090 11c177c8e97725db2631073c19f07b62  
b331b74b18dc79bcd6f6532d51e1637c1  
114091 66dea50a8b16d9b4dee7af250b4bela5  
edb027a75a1449115f6b43211ae02a24

|   | order_status | order_purchase_timestamp | order_delivered_customer_date \ |
|---|--------------|--------------------------|---------------------------------|
| 0 | delivered    | 2017-10-02 10:56:33      | 2017-10-10 21:25:13             |
| 1 | delivered    | 2018-07-24 20:41:37      | 2018-08-07 15:27:45             |
| 2 | delivered    | 2018-08-08 08:38:49      | 2018-08-17 18:06:29             |
| 3 | delivered    | 2017-11-18 19:28:06      | 2017-12-02 00:28:42             |
| 4 | delivered    | 2018-02-13 21:18:39      | 2018-02-16 18:17:02             |

...  
...  
114087 delivered 2018-02-06 12:58:58 2018-02-28 17:37:56  
114088 delivered 2017-08-27 14:46:43 2017-09-21 11:24:17  
114089 delivered 2018-01-08 21:28:27 2018-01-25 23:32:54  
114090 delivered 2018-01-08 21:28:27 2018-01-25 23:32:54  
114091 delivered 2018-03-08 20:57:30 2018-03-16 13:08:30

|   | order_estimated_delivery_date | customer_unique_id               |
|---|-------------------------------|----------------------------------|
| 0 | 2017-10-18                    | 7c396fd4830fd04220f754e42b4e5bff |
| 1 | 2018-08-13                    | af07308b275d755c9edb36a90c618231 |
| 2 | 2018-09-04                    | 3a653a41f6f9fc3d2a113cf8398680e8 |

|        |            |                                  |
|--------|------------|----------------------------------|
| 3      | 2017-12-15 | 7c142cf63193a1473d2e66489a9ae977 |
| 4      | 2018-02-26 | 72632f0f9dd73dfee390c9b22eb56dd6 |
| ...    | ...        | ...                              |
| 114087 | 2018-03-02 | da62f9e57a76d978d02ab5362c509660 |
| 114088 | 2017-09-27 | 737520a9aad80b3fbbdad19b66b37b30 |
| 114089 | 2018-02-15 | 5097a5312c8b157bb7be58ae360ef43c |
| 114090 | 2018-02-15 | 5097a5312c8b157bb7be58ae360ef43c |
| 114091 | 2018-04-03 | 60350aa974b26ff12caad89e55993bd6 |

|                  | customer_zip_code_prefix | customer_city           |
|------------------|--------------------------|-------------------------|
| customer_state \ |                          |                         |
| 0                | 3149                     | sao paulo               |
| SP ...           |                          |                         |
| 1                | 47813                    | barreiras               |
| BA ...           |                          |                         |
| 2                | 75265                    | vianopolis              |
| GO ...           |                          |                         |
| 3                | 59296                    | sao goncalo do amarante |
| RN ...           |                          |                         |
| 4                | 9195                     | santo andre             |
| SP ...           |                          |                         |
| ...              | ...                      | ...                     |
| ...              |                          |                         |
| 114087           | 11722                    | praia grande            |
| SP ...           |                          |                         |
| 114088           | 45920                    | nova vicoso             |
| BA ...           |                          |                         |
| 114089           | 28685                    | japuiba                 |
| RJ ...           |                          |                         |
| 114090           | 28685                    | japuiba                 |
| RJ ...           |                          |                         |
| 114091           | 83750                    | lapa                    |
| PR ...           |                          |                         |

|                        | review_id                        | review_score |
|------------------------|----------------------------------|--------------|
| review_creation_date \ |                                  |              |
| 0                      | a54f0611adc9ed256b57ede6b6eb5114 | 4.0          |
| 10-11                  |                                  |              |
| 1                      | 8d5266042046a06655c8db133d120ba5 | 4.0          |
| 08-08                  |                                  |              |
| 2                      | e73b67b67587f7644d5bd1a52deb1b01 | 5.0          |
|                        |                                  | 2018-        |

|        |                                  |     |       |
|--------|----------------------------------|-----|-------|
| 08-18  |                                  |     |       |
| 3      | 359d03e676b3c069f62cadba8dd3f6e8 | 5.0 | 2017- |
| 12-03  |                                  |     |       |
| 4      | e50934924e227544ba8246aeb3770dd4 | 5.0 | 2018- |
| 02-17  |                                  |     |       |
| ...    | ...                              | ... |       |
| ...    |                                  |     |       |
| 114087 | 29bb71b2760d0f876dfa178a76bc4734 | 4.0 | 2018- |
| 03-01  |                                  |     |       |
| 114088 | 371579771219f6db2d830d50805977bb | 5.0 | 2017- |
| 09-22  |                                  |     |       |
| 114089 | 8ab6855b9fe9b812cd03a480a25058a1 | 2.0 | 2018- |
| 01-26  |                                  |     |       |
| 114090 | 8ab6855b9fe9b812cd03a480a25058a1 | 2.0 | 2018- |
| 01-26  |                                  |     |       |
| 114091 | dc9c59b4688062c25758c2be4cafc523 | 5.0 | 2018- |
| 03-17  |                                  |     |       |

|        | review_answer_timestamp | seller_zip_code_prefix | seller_city \      |
|--------|-------------------------|------------------------|--------------------|
| 0      | 2017-10-12 03:43:48     | 9350.0                 | maua               |
| 1      | 2018-08-08 18:37:50     | 31570.0                | belo horizonte     |
| 2      | 2018-08-22 19:07:58     | 14840.0                | guariba            |
| 3      | 2017-12-05 19:21:58     | 31842.0                | belo horizonte     |
| 4      | 2018-02-18 13:02:51     | 8752.0                 | mogi das cruzeiros |
| ...    | ...                     | ...                    | ..                 |
| .      |                         |                        |                    |
| 114087 | 2018-03-02 17:50:01     | 17602.0                | tupa               |
| 114088 | 2017-09-22 23:10:57     | 8290.0                 | sao paulo          |
| 114089 | 2018-01-27 09:16:56     | 37175.0                | ilicinea           |
| 114090 | 2018-01-27 09:16:56     | 37175.0                | ilicinea           |
| 114091 | 2018-03-17 16:33:31     | 14407.0                | franca             |

|   | seller_state | product_category_name_english | delivery_time \ |
|---|--------------|-------------------------------|-----------------|
| 0 | SP           | housewares                    | 8.0             |
| 1 | SP           | perfumery                     | 13.0            |
| 2 | SP           | auto                          | 9.0             |
| 3 | MG           | pet_shop                      | 13.0            |
| 4 | SP           | stationery                    | 2.0             |

|        |     |                       |      |
|--------|-----|-----------------------|------|
| ...    | ... | ...                   | ...  |
| 114087 | SP  | baby                  | 22.0 |
| 114088 | SP  | home_appliances_2     | 24.0 |
| 114089 | MG  | computers_accessories | 17.0 |
| 114090 | MG  | computers_accessories | 17.0 |
| 114091 | SP  | health_beauty         | 7.0  |

|   |                       |
|---|-----------------------|
|   | average_delivery_time |
| 0 | 6.0                   |
| 1 | 3.0                   |
| 2 | 25.0                  |
| 3 | 20.0                  |
| 4 | 13.0                  |

|        |     |
|--------|-----|
| ...    | ... |
| 114087 | NaN |
| 114088 | NaN |
| 114089 | NaN |
| 114090 | NaN |
| 114091 | NaN |

[114092 rows x 35 columns]

order\_purchase\_timestamp

April 1984136.11

August 2099825.82

December 1045006.66

February 1658582.07

January 1591058.50

July 2073314.50

June 1878369.40

March 2006164.38

May 2218225.66

November 1590714.94

October 1105637.71

September 1016730.03

Name: payment\_value, dtype: float64

order\_purchase\_timestamp

April 9343

August 10843

December 5674

February 8508

January 8069

July 10318

June 9412

March 9893

May 10573

November 7544

October 4959

September 4305

Name: order\_id, dtype: int64

```

order_purchase_timestamp
2016-09      4
2016-10     324
2016-12      1
2017-01     800
2017-02    1780
2017-03    2682
2017-04    2404
2017-05    3700
2017-06    3245
2017-07    4026
2017-08    4331
2017-09    4285
2017-10    4631
2017-11    7544
2017-12    5673
2018-01    7269
2018-02    6728
2018-03    7211
2018-04    6939
2018-05    6873
2018-06    6167
2018-07    6292
2018-08    6512
2018-09     16
2018-10      4
Freq: M, Name: order_id, dtype: int64
product_category_name_english
bed_bath_table      11270
health_beauty       9727
sports_leisure      8700
furniture_decor     8415
computers_accessories 7894
...
arts_and_craftmanship    24
la_cuisine               14
cds_dvds_musicals       14
fashion_childrens_clothes 8
security_and_services    2
Name: count, Length: 71, dtype: int64
product_category_name_english
agro_industry_and_commerce 115852.18
air_conditioning           90267.59
art                        30693.05
arts_and_craftmanship      2326.17
audio                      59681.76
...
stationery                 314020.95
tablets_printing_image     9886.26

```

```

telephony          478496.89
toys               610357.60
watches_gifts     1407834.80
Name: payment_value, Length: 71, dtype: float64
product_category_name_english
agro_industry_and_commerce    265
air_conditioning              362
art                           230
arts_and_craftmanship         26
audio                         384
...
stationery                   2814
tablets_printing_image        89
telephony                   5166
toys                       4453
watches_gifts              6604
Name: order_item_id, Length: 71, dtype: int64
seller_id
7c67e1448b00f6e969d365cea6b010ab    506656.49
1025f0e2d44d7041d6cf58b6550e0bfa    309935.80
4a3ca9315b744ce9f8e9374361493884    300766.22
1f50f920176fa81dab994f9023523100    289362.32
53243585a1d6dc2643021fd1853d8905    276255.40
da8622b14eb17ae2831f4ac5b9dab84a    272636.60
4869f7a5dfa277a7dca6462dcf3b52b2    260333.69
955fee9216a65b617aa5c0531780ce60    235186.55
fa1c13f2614d7b5c4749cbc52fecda94    201638.28
7e93a43ef30c4f03f38b393420bc753a    181345.57
Name: payment_value, dtype: float64
seller_id
7c67e1448b00f6e969d365cea6b010ab    2.499814
1025f0e2d44d7041d6cf58b6550e0bfa    1.529206
4a3ca9315b744ce9f8e9374361493884    1.483963
1f50f920176fa81dab994f9023523100    1.427697
53243585a1d6dc2643021fd1853d8905    1.363028
da8622b14eb17ae2831f4ac5b9dab84a    1.345173
4869f7a5dfa277a7dca6462dcf3b52b2    1.284472
955fee9216a65b617aa5c0531780ce60    1.160397
fa1c13f2614d7b5c4749cbc52fecda94    0.994872
7e93a43ef30c4f03f38b393420bc753a    0.894749
Name: payment_value, dtype: float64
-0.30405831600767214
delivery_time
0.0      3.333333
1.0      4.429431
2.0      4.404308
3.0      4.372598
4.0      4.342077
...
191.0    1.000000

```

```

194.0    3.000000
195.0    1.000000
208.0    2.000000
209.0         NaN
Name: review_score, Length: 146, dtype: float64

```

```

import matplotlib.pyplot as plt
import seaborn as sns

# Time series plots
sales_trends_plot = revenue_trends.copy()

plt.figure(figsize=(10, 4))
plt.plot(sales_trends_plot.index, sales_trends_plot.values,
marker='o', linestyle='-', color='brown')
plt.title('Monthly Sales Trends')
plt.grid(True)
plt.ylabel('Revenue')
plt.show()

```



```

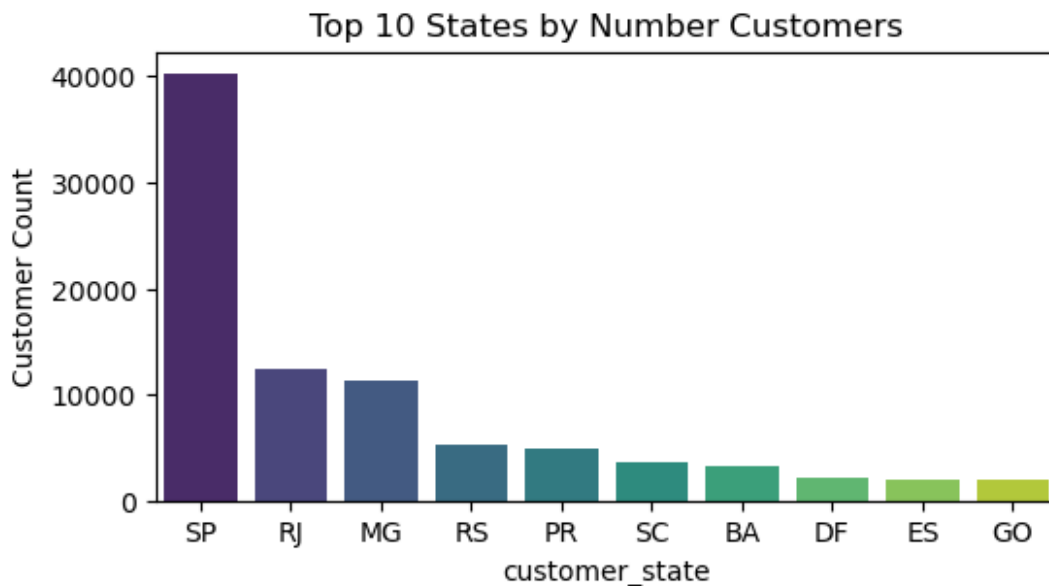
# Bar charts
import seaborn as sns
plt.figure(figsize=(6, 3))
state_dist = consolidated_dataset.groupby('customer_state')
['customer_unique_id'].nunique().sort_values(ascending=False).head(10)
sns.barplot(x=state_dist.index, y=state_dist.values,
palette='viridis')
plt.title('Top 10 States by Number Customers')
plt.ylabel('Customer Count')
plt.show()

```

C:\Users\Vinay\AppData\Local\Temp\ipykernel\_3236\3687707357.py:5:  
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=state_dist.index, y=state_dist.values,
palette='viridis')
```



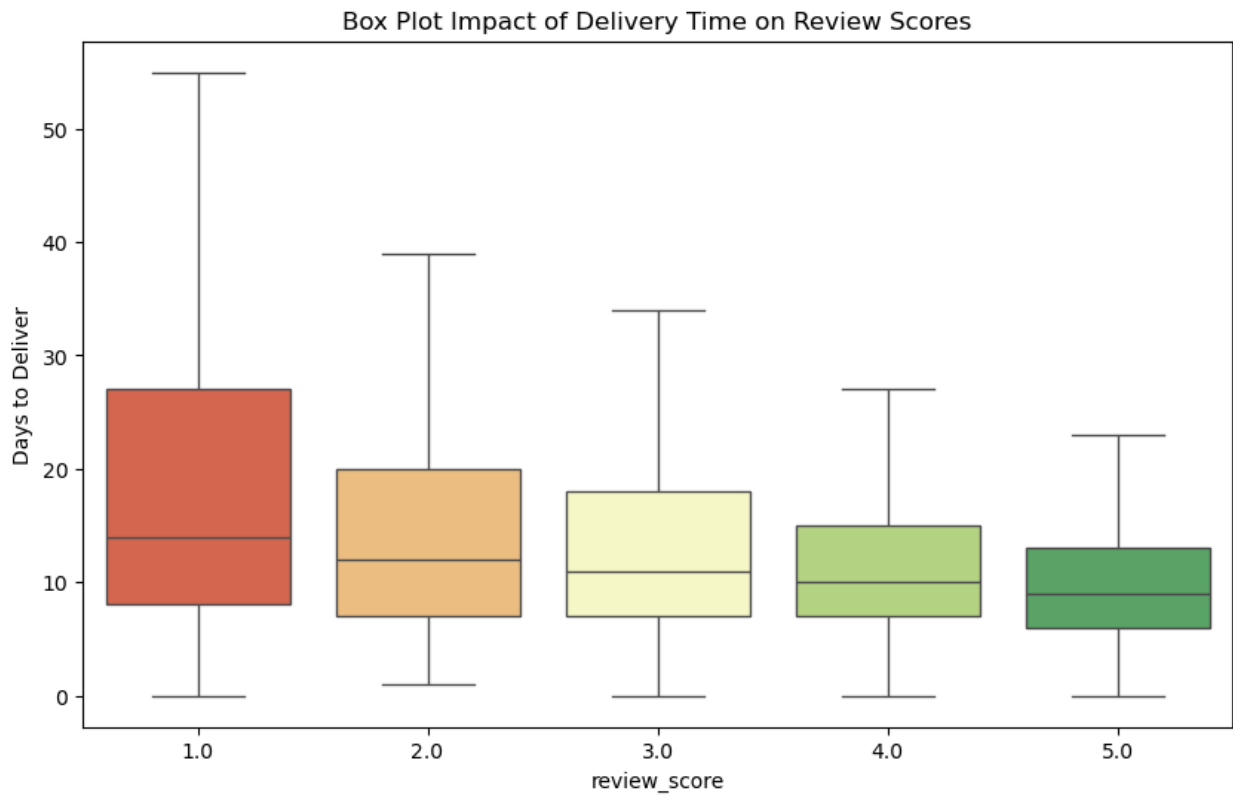
*# Box plot*

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='review_score', y='delivery_time',
data=consolidated_dataset, showfliers=False, palette='RdYlGn')
plt.title('Box Plot Impact of Delivery Time on Review Scores')
plt.ylabel('Days to Deliver')
plt.show()
```

C:\Users\Vinay\AppData\Local\Temp\ipykernel\_3236\3681346720.py:3:  
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x='review_score', y='delivery_time',
data=consolidated_dataset, showfliers=False, palette='RdYlGn')
```



```
# histogram  
plt.hist(consolidated_dataset['review_score'], bins=5)  
plt.show()
```

